ROB498 Design Report

Open-World Natural Language-Guided Semantic Mapping and Search with Micro-Aerial Vehicles

The Jinglers

Team Members:
Kevin Qu, Roderick Wu, Yang Xu, Allen Tao, Darrian Shue

Date: April 10th, 2026

Section 1. Design Overview

Modern warehouses stock a wide variety of products in large quantities across expansive facilities, and as a result inventory control consistently ranks as the biggest operational challenge for 62% of warehouse professionals [1]. Manual inventory tracking is slow, error-prone, and resource-intensive. Many tools that attempt to solve this issue have another problem: they are not user-friendly, and often require technical specialists to operate. Our project addresses both problems by developing an intelligent micro-aerial vehicle (MAV) platform that can autonomously map a warehouse environment, identify and localize objects within it, and allow non-technical users to interact with the system in natural language. This allows any individual to utilize our product with minimal training.

Our MAV solution operates in two stages. In the mapping stage, the MAV flies through the environment, building both a 3D occupancy map of obstacles (referred to as a voxel map from here onwards) and a semantic map that tracks the identities and locations of objects it detects. In the navigation stage, a user is able to interact with the MAV through a chat interface powered by a large language model (LLM). The user can ask questions about the environment, such as where a particular item is located or what objects are present, and can instruct the MAV to navigate to specific objects or positions. The MAV plans its path using the maps it has built, avoiding obstacles along the way.

Our team's design philosophy centered on building a minimum viable product first. Given the wide scope of the project, spanning depth estimation, 3D mapping, object detection, LLM integration, and motion planning all on limited hardware and compute, we recognized early on that trying to perfect individual components before integration would be risky within the design timeframe. Instead, we aimed to get each module functional and integrated end-to-end as quickly as possible, then iteratively improved components once the full pipeline was working (time permitting). To execute on this, we divided the work into parallel modules across our five team members. Two members focused on mapping, which included camera setup, stereo depth estimation, and voxel map construction. This was the most technically challenging portion of the project and represented the primary bottleneck. One member worked on semantic mapping and object tracking, one worked on LLM integration and motion planning, and the remaining member supported mapping efforts, setting up the learned depth estimation methods and data communication infrastructure. This parallel structure allowed us to make progress across all modules simultaneously while focusing on aspects where technical difficulty was greatest.

Section 2. Background and Related Work

The software and hardware frameworks that we opt for are dependent not only on our intended final design, but also the constraints we must work within. Our project incorporates elements from various active areas of robotics development, including semantic mapping, vision-based depth estimation, and natural language interfaces for autonomous systems. This section outlines some important technical foundations from which our design was built upon.

2.1 Semantic Mapping and Vision-Language Models

Traditional MAV systems are primarily designed to construct geometric maps of their surroundings, representing free space and obstacles. Semantic mapping extends this framework by assigning object identities and labels with their physical location, enabling a robot to reason about its

surroundings at a higher level [2]. This capability is essential for applications like warehouse inventory management, where a system must not only navigate safely but also know where specific objects are located. Recent advances in vision language models (VLMs) have opened new possibilities for integrating language understanding and perception into robotic systems. Several recent works have demonstrated the feasibility of using VLMs for UAV perception and planning [3][4], and others have explored using large language models as universal command and control interfaces for MAVs [5][6]. Our project takes inspiration from these ideas, but implements them in a more modular way. Rather than relying on a single end-to-end VLM for both perception and control, we used a dedicated object detection model for visual perception and a separate LLM for natural language interaction.

2.2 Object Detection

For object detection, we used YOLO-World, an open-vocabulary object detector that can detect objects from a user-specified list of categories (such as “umbrella”, “box”) built upon the YOLOv8 framework [7]. We initially considered using a full VLM for object detection, which would have offered more flexible scene understanding. However, VLM inference is substantially more computationally expensive, and YOLO-World provides a more balanced solution in terms of real-time performance. Furthermore, its open-vocabulary capability was important to us because a warehouse environment could contain a diverse and changing set of objects, so we didn’t want to limit ourselves to a fixed set of object classes.

2.3 Depth Estimation and 3D Mapping

Our system relies on stereo depth estimation to construct a 3D representation of the surrounding environment. Because the Intel RealSense T265 tracking camera provides only fisheye image pairs, we used an open source ROS2 package to rectify the stereo images and do classical depth estimation [8]. Later, we replaced classical depth estimation with Fast-FoundationStereo, a learned stereo estimation model which produced significantly better depth / point cloud maps [9]. To construct the map from the resulting point clouds, we used OctoMap, a well-established framework for 3D mapping that represents the environment as an octree-based voxel grid [10]. Implementing our MAV using these tools allowed our project to scale in complexity while remaining technically achievable.

2.4 Communication Infrastructure

A key architectural decision in our system was offloading computationally intensive tasks from the Jetson Nano onboard to an external laptop with a dedicated GPU. This required low-latency communication between the two devices. While ROS2 natively supports communication over networks, we found it to be too slow for streaming image data and instead adopted ZeroMQ (ZMQ), a lightweight messaging library that transmits data as raw bytes over a network socket.

2.5 LLM Integration and Tool Calling

For natural language interaction, we used the free Gemini 2.0 Flash model accessed through the OpenRouter API. We structured this interaction by using the tool calling paradigm, which is a standard pattern in LLM development supported as a core API feature by many major providers (OpenAI, Google,

Anthropic). In this design pattern, the model is provided with structured descriptions of all functions available to it, and can reason about when to call them based on user input. The model returns a structured JSON response indicating which function to call and with what arguments, allowing our local code to execute the corresponding operations on the MAV.

Section 3. Design Objectives and Solution Details

In our initial design proposal, we established a comprehensive set of high-level goals for our MAV system. While our overarching vision remained consistent throughout the project, we found it necessary to critically re-evaluate and rescope specific implementation strategies to meet strict hardware and time constraints.

Our foremost objective was the generation of an accurate semantic map. To achieve this, the MAV needed to successfully discover, classify, and localize objects within a scene with a high degree of autonomy and accuracy. Our original ambition was to engineer a system where the MAV could autonomously explore an unknown room to search for and map various objects. This level of semantic understanding is a fundamental requirement for the platform to execute downstream tasks. However, as the semester progressed, we recognized that fully autonomous exploration was not strictly required to demonstrate a minimum viable product. To ensure we could deliver a functional prototype within our allotted timeframe, we simplified this exploration requirement. The MAV now operates along a predefined, fixed square path, mapping the objects it encounters along this trajectory.

A significant portion of our design iteration centered on hardware capabilities and the absolute necessity of real-time processing. We initially intended to execute all semantic mapping algorithms entirely onboard the Jetson Nano. We quickly realized that achieving real-time performance strictly with on-board hardware was impossible given the compute limitations of our provided hardware and the computational weight of our mapping objectives. Because real-time mapping is critical for any practical application, we relaxed our strict onboard constraint. By opening up options to offboard the computational load, we were able to ensure that the system operated in real time. This pivot also better models a commercial product scenario, where engineers would likely have access to more powerful computing hardware than what was available for this prototype.

Building upon our mapping objectives, we also pursued a stretch goal to generate an accurate 3D occupancy voxel map of the environment. Generating this map proved exceptionally difficult because constructing reliable depth maps is inherently complex given the available camera (RealSense T265) and compute hardware. While our preliminary prototype could technically function without a voxel map, we decided to implement it regardless. We prioritized this feature because a 3D occupancy map significantly elevates the viability of a real-life product, allowing for much more sophisticated autonomous navigation.

Our second major high-level objective focused on ensuring the platform was both safe and robust during operation. This goal served as a forcing function, driving us to ensure our underlying mapping systems operated with high reliability. Beyond technical robustness, safety also mandated an intuitive user interface. We designed our system around the end-user, anticipating that warehouse workers utilizing this technology will likely be non-technical personnel who require a system that can be deployed easily without demanding specialized training. To facilitate this accessible interaction, we planned from the inception of the project to leverage a LLM. However, integrating an LLM introduced a significant technical hurdle in figuring out how to guarantee strict safety standards and reliable flight behavior when the MAV is tasked with interpreting and acting upon highly flexible, open-ended user commands.

Finally, our design process was heavily shaped by the reality of engineering within a single-semester undergraduate capstone course. We needed to ensure the project scope was tractable, resulting in a design simple enough to yield a feature-complete prototype within a reasonable timeframe. We also had to operate within strict hardware boundaries. For instance, our software needed to run efficiently enough to clearly demonstrate all core capabilities during brief flight windows, which were strictly constrained by the MAV's battery limits. We also set a design goal to keep the overall system affordable by actively minimizing both hardware and external compute costs. Because these constraints were non-negotiables, our team had to navigate numerous development hurdles and engineer creative workarounds using the provided hardware. Ultimately, we tailored our final demonstration to effectively highlight the core capabilities of our minimum viable product.

To formalize our system architecture, we decomposed our high-level design objectives into specific functional requirements (see figure 1 and table 1). This functional analysis guided our technical verification and established the metrics for our minimum viable product.

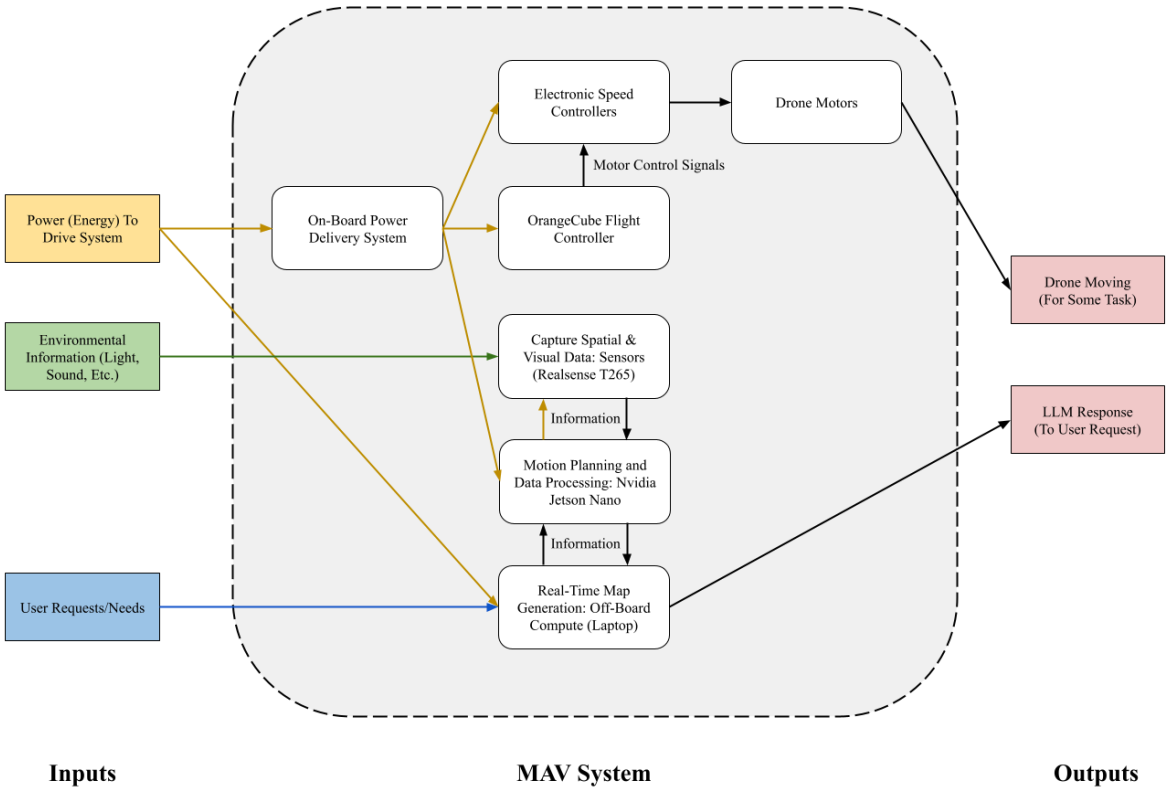


Figure 1. Functional analysis diagram.

Table 1: (Revised) Requirements Model

#	Objectives	Metrics	Criteria	Constraints
1	*HL1*: MAV should generate an accurate semantic map			
1.1*	MAV should localize objects accurately in the semantic map	1. RMSE of 3D pose of detected objects vs. Vicon ground truth	1. Higher is better	1. Must use the provided cameras
1.2*	MAV should discover and classify objects accurately in the semantic map	1. Percentage of objects correctly detected	1. Higher is better	1. Must use the provided cameras
1.3	Semantic mapping should run in real time	1. Update rate for object detection and classification [Hz]	1. Higher is better	1. Must run onboard the Jetson or with available compute (laptops)
1.4	MAV should track waypoints accurately	1. 3D Euclidean distance to target waypoint [m]	1. Lower is better	1. None
1.5	MAV should generate an accurate occupancy map	1. Precision and Recall of occupancy for each voxel 2. Resolution of voxel grid [m]	1. Lower is better 2. Lower is better	1. Must use the provided cameras
2	*HL2:* MAV should be safe and robust to operate			
2.1	MAV semantic mapping should be robust to environmental disturbances	1. Degradation in metrics in requirements 1.1, 1.2, 1.3, 1.4, 1.5	1. Lower is better	1. None
2.2	MAV user interface should be intuitive to untrained users.	1. Random user ratings/reviews and human robot interaction research	2. Higher is better	1. None
3	*HL3:* Project should be well-scoped for an undergraduate capstone course			
3.1*	MAV should be affordable	1. Fixed costs [CAD] 2. Variable costs (API calls, etc) [CAD]	1. Lower is better 2. Lower is better	1. Must use provided hardware in starter kit 2. None
3.2*	Feature-complete prototype should be completable within a tractable timeframe	1. Anticipated development time [hours]	1. Lower is better	1. Must be completable within allocated time slots for ROB 498
3.3*	MAV should be capable of flight for a reasonable duration	1. Total capacity of batteries [Wh] / Total power consumption [W]	1. Higher is better	1. Must be at least 1 minute to clear flight test 2 [6]

Section 4. Design Prototype Overview

Section 4.1 Systems Diagram

Our system consists of multiple interconnected parts that enable the advanced functions of voxel mapping, semantic mapping, object detection, natural language comprehension, motion planning and low-level control. Figure 2 illustrates a detailed block diagram of everything that enables this system.

At a high level, it fulfills the purpose of a warehouse management MAV by mapping out its environment in 3D with a semantic and a voxel occupancy map, and then allows a user without technical background to perform warehouse management tasks by querying and navigating the MAV through natural language.

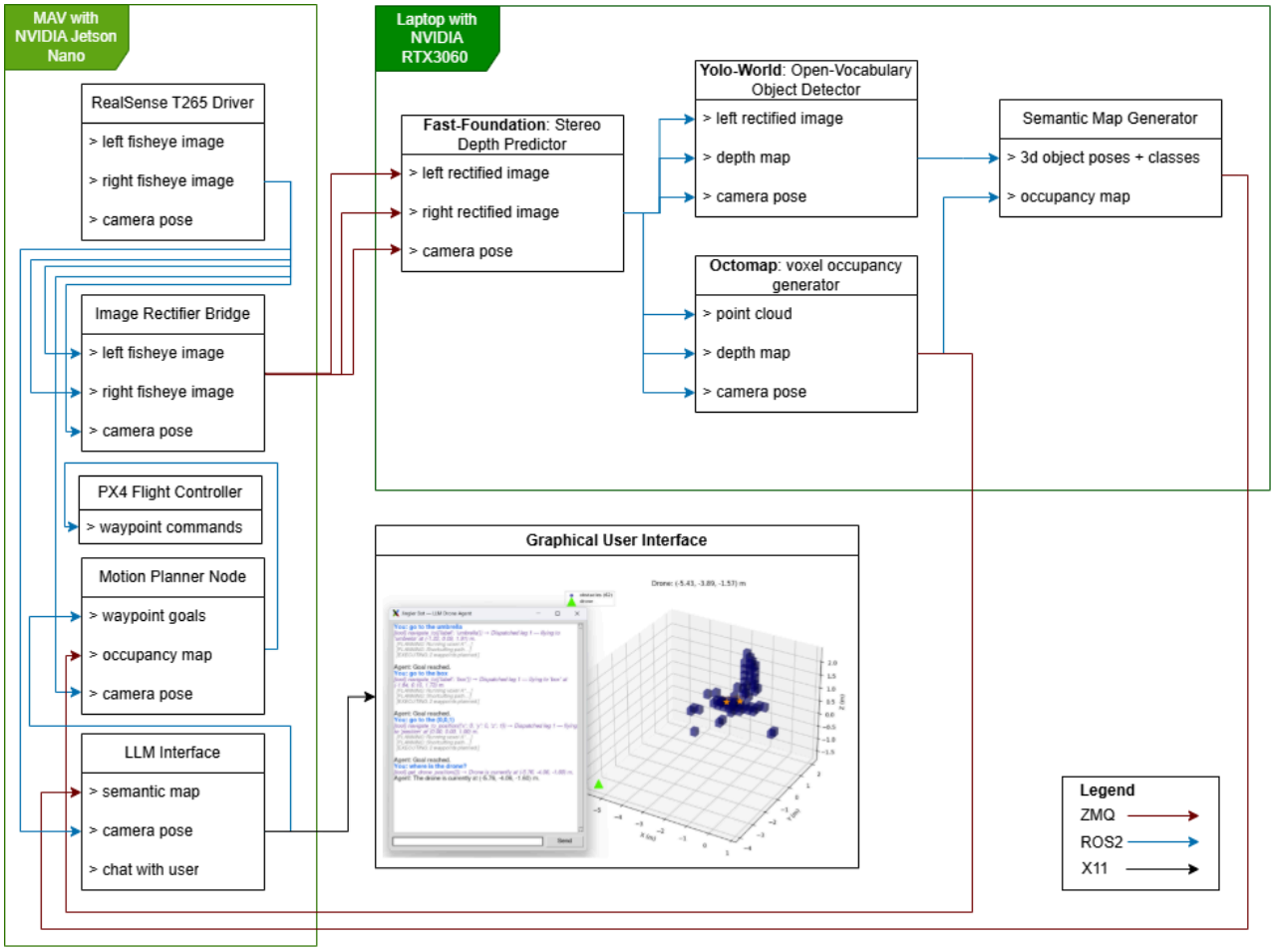


Figure 2. Block diagram of MAV-offboard computer system

The compute for our system is split into two distinct phases: lightweight onboard computations on the MAV for real-time features that operate close to the sensors and actuators of the MAV, and offboard computation for intensive tasks that are more resilient to frame dropping.

Section 4.2 System Implementation

On the MAV, we rely heavily on the RealSense T265 for both localization and visual image data. Out of the box, this stereo camera system is a good localizer but has poor image quality, as it only outputs low-res, fisheye-distorted stereo image pairs. Due to the constraints of the project, we were unable to use a dedicated stereo camera system such as the D435, which is capable of generating high-fidelity point clouds. Our system starts with the RealSense camera driver and utilizes the `t265_depth` ROS2 package [8] to perform image rectification for stereo depth estimation. This package provides both rectified images and an implementation for a tunable semi-global block matching algorithm for depth estimation. However, we found that the depth maps from this algorithm were both unreliable and inconsistent; many areas were laden with speckling artifacts. Exploring other more reliable techniques, we found NVIDIA's Fast-FoundationStereo model for learned stereo depth estimation [9]. Although not perfect, the model's outputs were much more reliable than the classical algorithm. However, it still faced issues with output inconsistency from frame to frame and poor performance on reflective surfaces like the windows in the flight arena.

When deploying the model, despite the Jetson Nano's onboard CUDA device, its capabilities were far too weak to run the model. Even a single inference would crash the whole system due to exceeding RAM limits, and the Jetson is not equipped to support PyTorch 2.x, which is a requirement of the inference library.

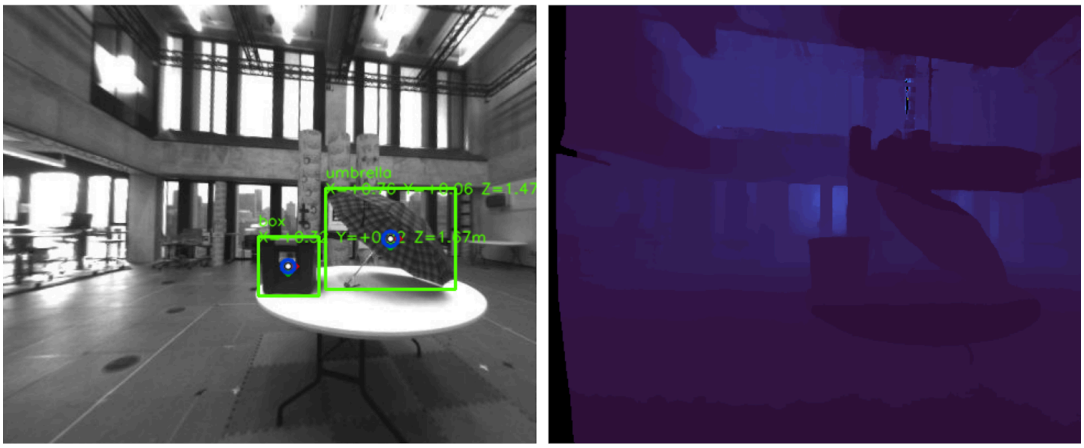
We decided to move all computationally expensive operations like depth estimation to one of our members' laptops, which is equipped with an NVIDIA RTX3060 GPU. This added an additional layer of communication to our system. Although ROS2 over the network works for passing messages between the MAV and laptop, it is extremely slow in sending compressed images, and frequently drops messages. Instead, we implemented a different messaging protocol, ZMQ. ZMQ connects the MAV to the laptop using a network socket and sends all the message data in a dense, raw-bytes format. We found that this ZMQ protocol was able to stream at a consistent rate of 2Hz. This speed is sufficient for our purposes since the inference and voxel map are limited by a maximum of 5Hz, and the map updates operate well at 2Hz for our uses. To ensure that the 2Hz rate would allow us to generate a reliable semantic and occupancy map, we enforced an upper limit of 0.5 m/s on the MAV's translational velocity and 30 deg/s on the rotational velocity through QGroundControl.

By handling message passing using our own code, we were also able to timestamp-match poses and rectified images together, ensuring a max slop of 0.01s in downstream tasks. On the laptop, the ZMQ messages were republished as ROS2 messages.

The depth estimation model has two inference options: PyTorch and TensorRT. By deploying on TensorRT, we were able to cut our inference time for each frame to just 35 ms, down from 115 ms. The TensorRT deployment required our input shapes to be in multiples of 32, so we had to further tune our camera intrinsics to account for this crop when generating point clouds.

We perform all our semantic mapping and object tracking tasks using YOLO-World, which is an open-vocabulary object detector. By passing in a list of objects we are interested in tracking, the model is able to determine a bounding box for all objects that match that label within the scene. All image processing is done from the left camera's rectified image. We then map the detected position of the object in the 2D camera image onto the depth map to retrieve the depth of the object. This local object pose is then transformed into the global frame and tracked using an Exponential Moving Average smoothing function. Tracks of the same class that are close to each other are merged. Figure 3a shows the YOLO

world output with the associated learned depth map of the same frame, and 3b shows stable tracks along with the MAV path and pose.



a) YOLO-world detections and depth map output for a single frame

b) Bird's eye view of MAV path, pose and tracked objects

Figure 3. YOLO World detections with depth maps and associated object tracks

Using the camera intrinsics, we generate a point cloud from the depth map. This point cloud is transformed into the global frame and is passed to OctoMap to generate a voxel occupancy map [10]. A handful of parameters were tuned to improve the map quality. In our final version, we settled on a voxel size of 0.25m.

As the final stage of the computations that run on the laptop, the tracked object locations are tagged to occupied voxels from the OctoMap output. This final semantic map and occupancy grid is then packed into a raw-bytes message to be sent back to the MAV over ZMQ.

Just like how the laptop accepts timestamp-matched ZMQ messages to unpack and retransmit over ROS2, the MAV does the same for every semantic map and voxel occupancy map it receives. The semantic map information is passed to our agentic LLM interface through a chat window that shows on the user's screen.

Our LLM interface operates much like other agentic frameworks used in other modern language model applications. We run our LLM on the cloud using the OpenRouter API and use the Gemini Flash 2.0 model. The user's prompt is appended to a system prompt to indicate to the model that it has agency over a quadcopter MAV with a set of predefined operations. Additionally, the model is passed the semantic map dictionary and the MAV's current position to give it spatial understanding. We implement four basic primitives: navigate to object, list objects, navigate to position and query the MAV position. The LLM responds to prompts with a call to one of these four actions. The actions demonstrate the capability of navigating to an object in the semantic map, listing out the full semantic map with object tags and locations, navigating to an arbitrary (but bounded) position, and querying the MAV's position. Since we are interfacing with a language model, it supports arbitrary user prompts and can respond in natural language. For example, in a scene with just an umbrella and a box, prompting the model with "go to the object you would use when it is raining" would result in triggering the tool to navigate to the umbrella's semantic map position. A prompt such as "where is the apple?" would result in the model saying "There is no apple in the scene." Users can also chat with it through natural language as with any LLM. Figure 4 shows a sample run where the user asked navigation and query commands. It also shows a visualization of the voxel map and semantic map.

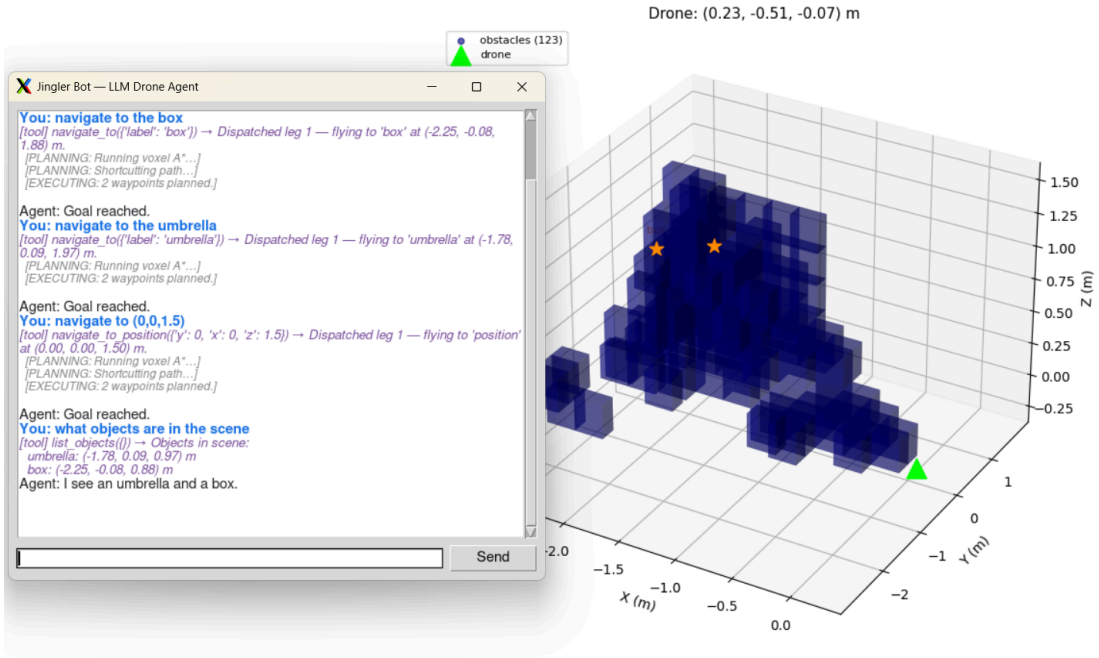


Figure 4. Sample chat history with LLM agent along with voxel map + semantic map visualization. The MAV's location is indicated with a triangle and the tagged object's locations are indicated with the stars.

Finally, our MAV implements a path planning algorithm. All occupied voxels along with a safety margin of 1 voxel around them are considered obstacles in our algorithm. Every navigation call will run a 3D A* shortest path plan to a target point while avoiding all obstacles.

Section 4.3 Scaling to Production

In our prototype, we demonstrate the MAV's capability of mapping a simple environment through a predefined mapping route that it takes to generate the semantic and voxel maps. In the flight arena environment, this was one full loop around our scene. However, in a real deployment, we could leverage autonomous mapping and planning algorithms to generate the map of the environment/warehouse.

The LLM interaction portion of our project remains the same and fully functional as it is; it can be scaled to a production environment. We could add additional features in this phase such as stateful inventory tracking across runs, but our current features are sufficient in supporting basic warehouse management.

Section 5. Results and Evaluation

Section 5.1 Evaluation Setup

To perform quantitative evaluation against our objectives in table 1, we collected a scenario with Vicon ground truth. In figure 5 below, we affix Vicon markers to the center top of 3 evaluation objects below. To then determine the Cartesian pose of each object, a measuring tape is used to measure the x, y, and z distance from the marker to the center of the object, and that offset is applied to generate the ground truth pose for each object in the world frame. After recording the ground truth pose for each object, and assuming a static scene, the Vicon marker is transferred to the MAV. The MAV is then operated to capture a rosbag of the whole scene. The generated semantic map for its object classes and their corresponding 3D poses are directly evaluated against the object poses obtained earlier.

Figure 5. Vicon Markers on Evaluation Objects

Section 5.2 Verification

Section 5.2.1 Semantic Map

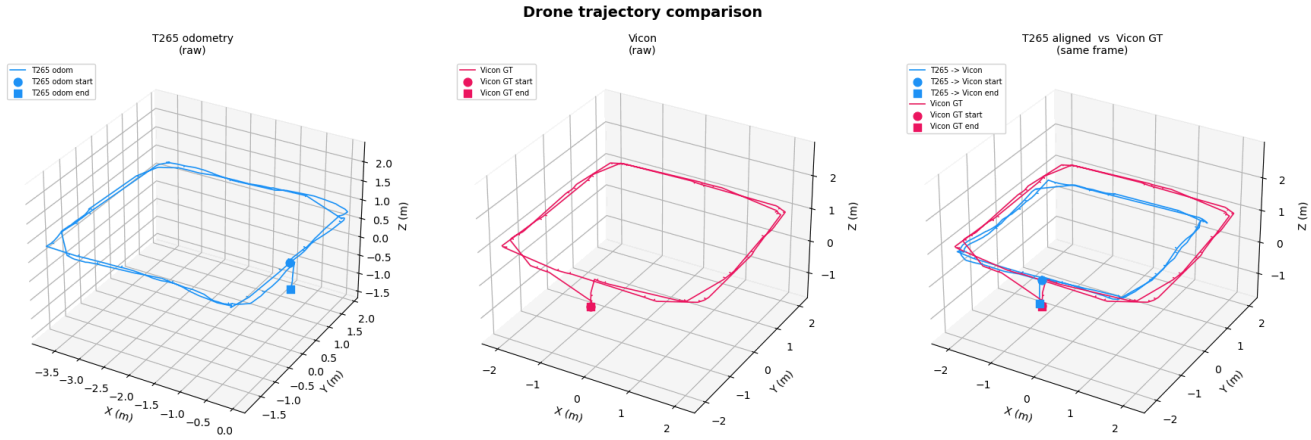


Figure 6 Camera Pose vs. Vicon Pose (GT) for the Mapping Route

Since the MAV is designed to map out any warehouse, we cannot assume an existing positioning system. Therefore, we first evaluate the quality of pose data from the onboard RealSense T265 Camera. We evaluate this pose against the Vicon pose of the MAV in the evaluation sequence. In figure 6, the blue plot denoting the RealSense data exhibits a systematic undershoot of 0.35 meters compared to the red plot representing the Vicon pose when tracing out a 2x2 meter square. This error is propagated downstream to the semantic 3D object detections and the voxel map.

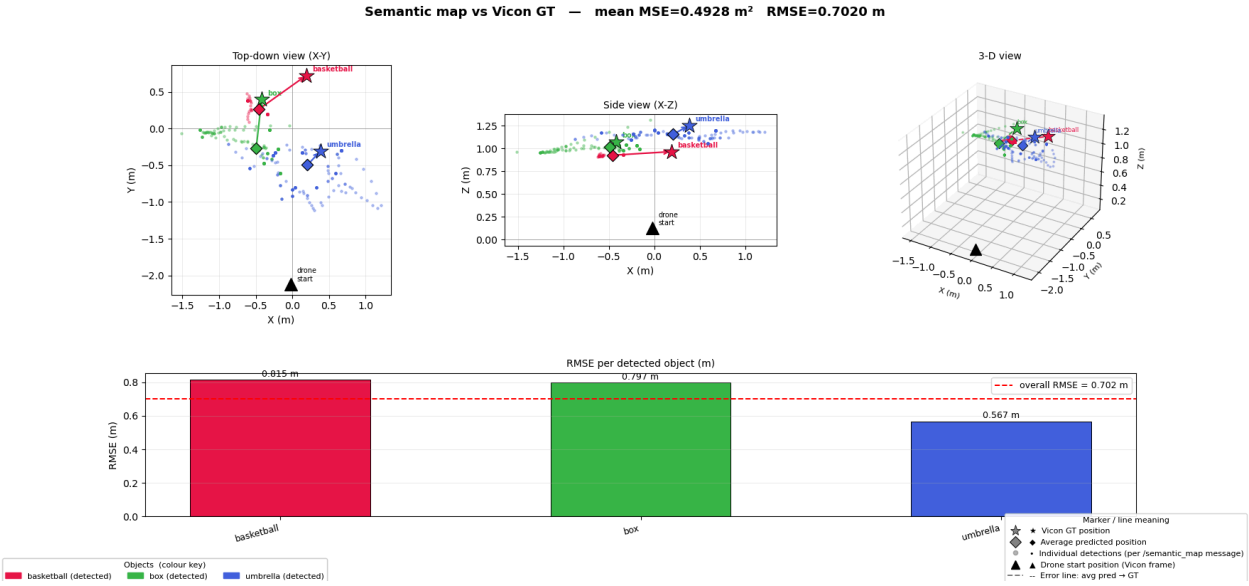


Figure 7. Semantic Mapping 3D Object Poses vs. Vicon GT

To evaluate the quality of open-vocabulary object detections from YOLO-World, we plot the YOLO detections transformed to world frame using the MAV pose and compare it to the ground truth

pose with each Vicon object. Results are illustrated in figure 7. Specifically, there exists an overall root mean squared error of 0.7020 meters over three detected objects (basketball, box and umbrella). Triaging where these errors stem from, we notice that the individual object detections, denoted by the dots in figure 7, exhibit high variance, varying up to 1 meter in the x and y dimensions. This high source of random error is likely due to depth ambiguities. Since the 2D detections are lifted into 3D using the depth values within the bounding box, said depth values only capture the surface of the object. When the viewing angle to the object changes as the MAV progresses in its path, it captures a new surface, and therefore proposes a new 3D pose centered on the new visible face. Additional sources of error are due to jitter in the Fast-FoundationStereo depth maps, introducing further random error. Finally, the perception pipeline was able to pick up 3 of 4 objects, failing on the football. Due to the limitations in hardware in adopting the T265, notably the relegation to fisheye and greyscale images, small objects pose a challenge in recall to the open-vocabulary YOLO-World pipeline.

Section 5.2.2 Runtime

Table 2: Runtime of major submodules

Module	**Max Inference Throughput**
Fast-FoundationStereo	28 Hz
YOLO-World	37 Hz
OctoMap	5 Hz
ZMQ image streaming	2 Hz
T265 Rectified Image Bridge	25 Hz

The maximum inference speed of all major modules outlined in figure 2 are detailed in table 2. Notably, there exists a 2 Hz bottleneck during the image streaming from the MAV to the laptop, with all other modules achieving higher throughput thanks to offloading computation to a NVIDIA RTX 3060-equipped laptop and TensorRT. As our MAV obeys a velocity limit of 0.5 m/s, the max error is 0.25m, which corresponds to our voxel size. Therefore, inflating the voxel map by 1 voxel will enable the MAV to avoid mapped obstacles.

Section 5.2.3 Occupancy Map

To evaluate the quality of the generated voxel occupancy map, a ground-truth occupancy grid must be generated for the evaluation scene. To proxy the ground truth, the scene contains 3 large pillars shown in figure 8. A Vicon marker is affixed at the top of the center pillar. Then, for each ground truth object in figure 5, as well as the pillar in figure 8, a measuring tape was used to measure the object dimensions. Then, the entire volume from the ground to the objects' respective Vicon marker was filled and marked as an occupied voxel in ground-truth in the Vicon frame.

To evaluate the generated occupancy map against the ground truth map, our occupancy map is first transformed into Vicon space, using the T265 pose to Vicon transformation available thanks to the Vicon marker affixed to the MAV. Then, precision and recall metrics were computed for classifying each

voxel as occupied or unoccupied. An illustration is shown in figure 9, where black voxels denote occupied ground-truth voxels, green voxels are true positives, and red voxels are false positives.

The large number of false positives are predominantly due to two factors. First, we add a one voxel equivalent of padding on top of the predicted occupancy map, to account for the max error incurred due to latency at our MAV's flight speed, which was 0.25m, equal to our voxel size. Second, the depth map from Fast-FoundationStereo, though vastly superior to semiglobal block matching, still exhibits spurious inconsistencies, especially for surfaces such as glass windows present in the arena. We expect the overall precision and recall of 37% and 53% to be improved upon upgrading to a standard stereo depth camera designed for depth map outputs, like the RealSense D435. Nonetheless, the existing performance suffices for proof of concept in simple scenarios.

Figure 8. Vicon Marker Affixed on Top of Pillar

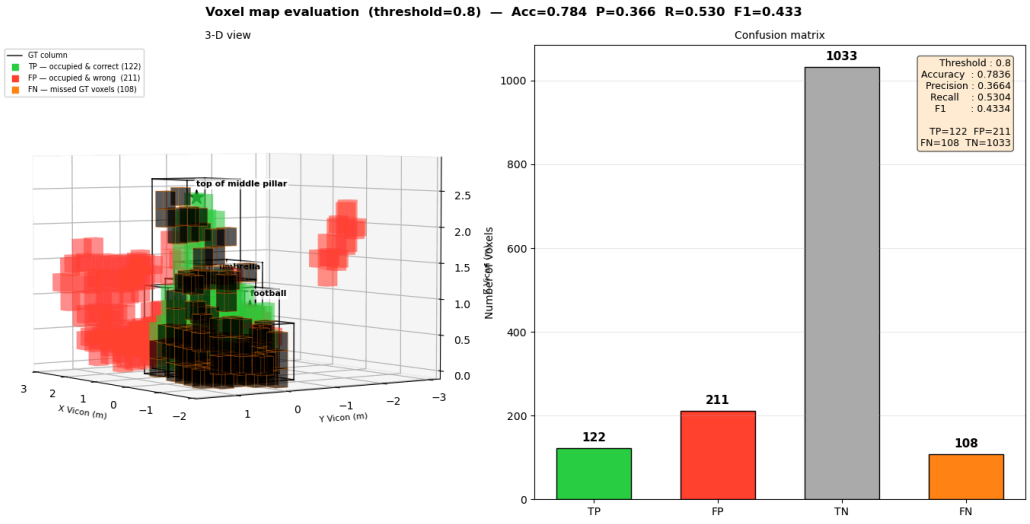


Figure 9. Evaluation Metric for Occupancy Mapping

Section 5.2.4 Flight Time

The MAV can maintain 4 minutes of continuous flight. Notably, the speed limits imposed on the MAV of 0.5 m/s of translational velocity and 30 deg/s of rotational velocity limit the operational coverage of the MAV. This is due to constrained compute and sensor hardware requiring a slow mapping run to generate a map of sufficient fidelity. Therefore, the max operational distance is 0.5 m/s × 4 mins × 60 sec/min = 120 meters in mapping an area.

Section 5.3 Validation

An LLM provides a natural language interface to minimize the technical know-how required to operate the MAV. Works on LLM human robot interfaces have demonstrated that adopting natural language as an interface represents a lower barrier-of-entry shift to an intuitive, guidance and example-driven approach for interacting with robots [11]. As our LLM offers a chat window as a layer of abstraction between low-level MAV skills (e.g. navigate to waypoint, ego-position, semantic map queries), it improves the usability of the system for an untrained user.

Validation for the adoption of a fully standalone MAV equipped with its own positioning and mapping system is performed through secondary research with existing unfulfilled warehouse needs. Warehouses average 180,000–200,000 sq. ft. and up to 40+ feet in height [12]. Thus, a MAV provides the agility to cover the entire space. Moreover, it is often implausible with regard to cost to install a positioning system that covers the entire warehouse space (e.g. Vicon), therefore the extra technical challenge of onboard localization on the MAV is well justified as a design requirement in table 1.

Section 5.4 Results and Discussion

Taking stock, our semantic mapping system is functional for mapping out large objects, working consistently for a large tupperware box and an open umbrella, and providing a functionally complete demonstration of occupancy mapping and semantic mapping for these two objects placed on a table, with the MAV traversing a 2x2 meter square path. The localization error of 0.35 meters and the 3D pose error of the lifted YOLO-World detections of 0.7020 meters enable sufficient granularity to navigate to detected objects, provided they are sufficiently spaced out and large.

Being provident of these limitations, we designed our MAV to be maximally modular, such that clean upgrade paths that directly improve precision are simple to implement without modifying any existing modules. For instance, all perception inputs are taken from one RealSense T265 camera. A RealSense D435 camera serves as a drop-in replacement with built-in depth maps and point clouds to immediately improve voxel occupancy mapping and depth for lifting YOLO-World detections. Similarly, a more performant Jetson, such as the Jetson AGX Orin, enables the lightweight Fast-FoundationStereo and YOLO-World models to be inferred solely onboard, without incurring the latency penalty of shuffling images over WiFi.

Section 6. Lessons Learned

This section outlines the technical and non-technical challenges that we faced throughout the design process. This resulted in specific design choices, adjustments, and accommodations that we made.

Section 6.1 Technical Challenges

There were a number of technical challenges that we encountered throughout the design process. These difficulties resulted in changes to the project's original proposed structure or forced us to make certain adaptations. Recalling the system's design, we encountered technical difficulties within the function of each individual system as well as during the orchestration of the various components together.

Starting from the highest level of abstraction, our MAV is controlled by the LLM called through OpenRouter. We experienced issues with the performance of the available free models when interacting with our tools. Since language model output is not deterministic, weaker models occasionally failed to output the correct json format, resulting in failed tool calls. These were solved by incorporating fail-safes and other "band-aid" fixes commonly used in agentic AI tools. For example, this often included re-prompting upon failures and specific text outputs.

The mapping system was especially problematic. Initially, we planned on using the D435 stereo camera for mapping. The camera provides a point-cloud, which can be trivially converted to a voxel occupancy map. However, the constraints of the project forced us to use more complex solutions. Using the rectified stereo images provided by the RealSense T265, it is possible to manually obtain a point-cloud by performing stereo matching algorithms to obtain disparity maps. It is possible for this to perform to a sufficient accuracy for generating useful point-clouds. However, we were unable to tune the process to be reliable enough, hence the switch to learned disparity methods. Learned methods, although more accurate, included their own set of challenges. As mentioned prior, the slow inference time necessitated that we instantiate a separate ROS instance on a laptop to run the learned model on TensorRT, which also required implementing ZMQ to maintain adequate communication speed. Once the system was polished to the final state, the learned disparity mappings still presented a major problem: although the disparity maps were individually clear, they were not consistent with each other. For example, two pairs of stereo-images taken consecutively in time should differ by very little, but we did not find this to be the case. When mapping the pixel disparities to 3D environment points, we found that although the point-cloud generated by a single stereo-image pair appeared to be very accurate, the combined point-cloud generated from a set of consecutive images was very noisy and often unrecognizable from the true environment. This result was only slightly superior to the point-clouds generated by traditional stereo-matching algorithms. For our final design, the voxel map was carefully tuned to adjust for these issues from the generated point-clouds, but is ultimately still very flawed for this reason. The difficulty of manually setting up the voxel occupancy map pipeline from a camera not designed to produce depth maps in the first place is one aspect of our design we would advise future teams to avoid pursuing if there are no prebuilt frameworks (e.g. D435 point-clouds).

YOLO-world inference was another driving factor behind choosing to use ZMQ with a separate ROS instance in order to offboard computation from the Jetson. Outside of ensuring that the model could run, there were also challenges with the object detection process. We encountered cases of our YOLO-world model detecting multiple instances of an object. We fixed this by merging same-class instances that were close together in 3D space.

Section 6.2 Non-Technical Challenges

Most of our non-technical challenges focused on meeting deadlines on time. Although we have fewer class hours than in earlier years of our degrees, our team members continue to have other time-consuming commitments such as thesis work. Furthermore, thesis working hours are not consistent

across group members, and often fluctuate at any given time. This presented a challenge in identifying availability for group work, especially during periods of integration, where additional meeting time was required, and all members had to be present. In the future, it might be beneficial to use third-party planning tools (e.g. LettuceMeet), or set up regular weekly meetings and/or work time.

Furthermore, when we assign different work to group members, the variance in difficulty/speed of the tasks is unclear and can lead to idle time where some team members are waiting for others to complete their components before moving on to next steps. This is a difficult challenge to address since there are no simple solutions as redistributing work between members often creates challenges for members who are new to a task. This presented a lesson in the natural friction of group work, forcing us to adapt as group dynamics evolve, such as pair programming to unblock the blocked team member.

Section 6.3 Reflection

For future capstone projects, we would advise that groups pursue simpler projects that have been shown to work before. Novelty can come from small adjustments or design choices, but stringing together so many ideas altogether was found to be exceedingly difficult. If we were to create another capstone project, we might choose a slightly less ambitious task. Perhaps like many of our peers, we would create a MAV that follows a moving target on the ground.

Section 7. Conclusion

This project successfully developed and demonstrated a minimum viable product for an autonomous, natural-language-controlled warehouse inventory micro-aerial vehicle (MAV). By integrating an LLM-driven interface with semantic and 3D voxel mapping, we proved that complex robotic operations can be abstracted away, allowing non-technical personnel to manage warehouse inventory through intuitive, conversational commands. The system successfully mapped a test environment, identified and localized open-vocabulary targets, and navigated autonomously to user-requested objects.

The most significant engineering challenges we encountered were rooted in hardware constraints. Specifically, the compute ceiling of the Jetson Nano and the visual limitations of the RealSense T265 camera. To achieve real-time performance, we successfully pivoted to a modular, offboard compute architecture. By utilizing a ZMQ pipeline for low-latency data transmission, we bypassed the onboard compute bottleneck. This allowed us to leverage TensorRT and a dedicated GPU to run perception models that could not be run on the Jetson, like YOLO-World (at 37 Hz) and Fast-FoundationStereo (at 28 Hz), alongside OctoMap for voxel generation.

While the final prototype achieved its functional requirements, it did exhibit a localization error of 0.35 meters and a 3D pose error of 0.7 meters. These inaccuracies largely stem from the T265's fisheye, greyscale images and the inherent noise of stereo depth estimation on this hardware. However, because we designed the software architecture to be strictly modular, these are hardware limitations rather than fundamental framework flaws. Drop-in hardware upgrades such as a RealSense D435 for native point clouds or a Jetson AGX Orin for entirely onboard inference would immediately resolve these accuracy and latency issues without requiring any algorithmic overhauls.

Ultimately, balancing the ambitious scope of integrating advanced perception, 3D mapping, and agentic AI within a single-semester capstone presented steep technical and scheduling hurdles. Despite these challenges, the resulting prototype is both feature-complete and highly scalable, demonstrating a viable, user-centric blueprint for the next generation of autonomous warehouse robotics.

Section 8. Supporting Materials

https://github.com/fishingguy456/foundation_stereo_ros
https://github.com/allenapplehead/t265_depth
https://github.com/allenapplehead/jinglers

Section 9. References

[1] K. Moore, “The Top 12 Warehouse Challenges: A Study of 100+ Warehouse Leaders,” *Kardex.com*, Jun. 04, 2025. https://www.kardex.com/en-us/blog/warehouse-challenges

[2] S. Raychaudhuri and A. X. Chang, “Semantic Mapping in Indoor Embodied AI -- A Comprehensive Survey and Future Directions,” arXiv.org, 2025. https://arxiv.org/abs/2501.05750.

[3] Y. Tian et al., “UAVs meet VLMs: Overviews and perspectives towards agentic low-altitude mobility,” Information Fusion, vol. 122, p. 103158, Apr. 2025, doi: https://doi.org/10.1016/j.inffus.2025.103158.

[4] F. Mehboob, M. James, A. Habel, J. Sam, M. A. Cabrera, and D. Tsetserukou, “MAVVLA: VLA based Aerial Manipulation,” arXiv.org, 2026. Available: https://www.arxiv.org/abs/2601.13809. [Accessed: Feb. 14, 2026].

[5] J. N. Ramos-Silva and P. J. Burke, “A Universal Large Language Model -- MAV Command and Control Interface,” arXiv.org, 2026. https://arxiv.org/abs/2601.15486 (accessed Feb. 13, 2026).

[6] “Cognitive Drone: VLA Model & Benchmark,” Github.io, 2025. Available: https://cognitivedrone.github.io/. (accessed Feb. 14, 2026).

[7] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, “YOLO-World: Real-Time Open-Vocabulary Object Detection,” *arXiv.org*, Feb. 22, 2024. https://arxiv.org/abs/2401.17270

[8] P. De Petris, "t265_depth," GitHub, 2021. [Online]. Available: https://github.com/tiralonghipol/t265_depth. Accessed: Apr. 9, 2026.

[9] B. Wen, S. Dewan, and S. Birchfield, “Fast-FoundationStereo: Real-Time Zero-Shot Stereo Matching,” *arXiv.org*, 2025. https://arxiv.org/abs/2512.11130 (accessed Apr. 09, 2026).

[10] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, “OctoMap: an efficient probabilistic 3D mapping framework based on octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, Feb. 2013, doi: https://doi.org/10.1007/s10514-012-9321-0.

[11] C. Wang et al., “LaMI: Large Language Models for Multi-Modal Human-Robot Interaction,” arXiv.org, Apr. 11, 2024. https://arxiv.org/abs/2401.15174 (accessed Apr. 30, 2024).

[12] painterwh, “Warehouse Size Guide: What’s the Average Warehouse Size?,” *Warehouse Logistics NYC*, Oct. 21, 2025. https://warehouse-logistics-nyc.com/how-big-is-the-average-warehouse/